Banking Transaction & Account Management System

Final Project Report

Course: MTM4692 – Applied SQL
Instructor: Fettah KIRAN

Team Members

Name	Student ID
Aykut ANLAK	22058606
Gökay BAHTİYAR	21058048
Umut Berke AYGÜNDÜZ	21058019
Uras AMAN	21052064
Yusuf Doğan GÜNEŞ	22052008

GitHub Repository:
https://github.com/aygunduzumutberke/banking-database-project

1 1. Introduction & Problem Statement

In the contemporary financial landscape, banking institutions generate a massive, continuous stream of interconnected operational data. Every single day, countless customer interactions, account updates, branch allocations, loan issuances, and multi-channel transaction flows take place simultaneously. Managing this vast ecosystem without a carefully structured architecture inevitably leads to operational bottlenecks. Without a robust relational framework, critical tasks—such as monitoring customer compliance, tracing fund velocities, auditing branch efficiencies, and identifying anomalous account behaviors for fraud detection—become exceptionally difficult and error-prone.

To bridge this gap, this project delivers the **Banking Transaction & Account Management System**, a fully normalized relational database engineered on SQLite. The system transitions raw, chaotic financial activities into a clean, queryable, and highly consistent environment. Beyond simple data persistence, this implementation serves as an analytical engine designed to empower stakeholders with reliable, data-driven reporting and operational visibility.

2 2. Project Objectives

The core architectural and analytical goals of this project are:

- **Normalized Relational Design:** Constructing a clean database blueprint that eliminates data redundancy while maintaining absolute integrity across all banking business entities.
- **Rigorous Relationship Mapping:** Establishing strong data consistency through accurate Primary Key (PK) and Foreign Key (FK) constraints.
- **Realistic Data Synthesis:** Populating the schema with realistic, synthetic financial records to simulate authentic banking behaviors and workloads.
- **Advanced SQL Analytical Exploitation:** Demonstrating the practical utility of complex SQL techniques—including multi-table joins, aggregations, subqueries, Common Table Expressions (CTEs), views, and indexing—to solve real-world financial reporting challenges.

3 3. Database Design & Schema Explanation

The core architecture is built around seven highly cohesive tables: `customers`, `branches`, `accounts`, `transactions`, `cards`, `loans`, and `payments`. At the absolute center of this ecosystem sits the `accounts` table, acting as the fundamental operational bridge. While customers and branches establish the baseline ownership, all dynamic financial activities—such as card operations, transactional histories, and payment logs—are directly anchored to specific accounts.

3.1 Entity Breakdown:

- **Customers:** Safeguards identity details and demographic footprints (`first_name`, `last_name`, `national_id`, `city`, etc.).
- **Branches:** Maps physical banking infrastructure, storing branch locations and identification markers.
- **Accounts:** Tracks the financial status, holding balance sheets, currencies, account classifications (e.g., checking vs. savings), and operational states.

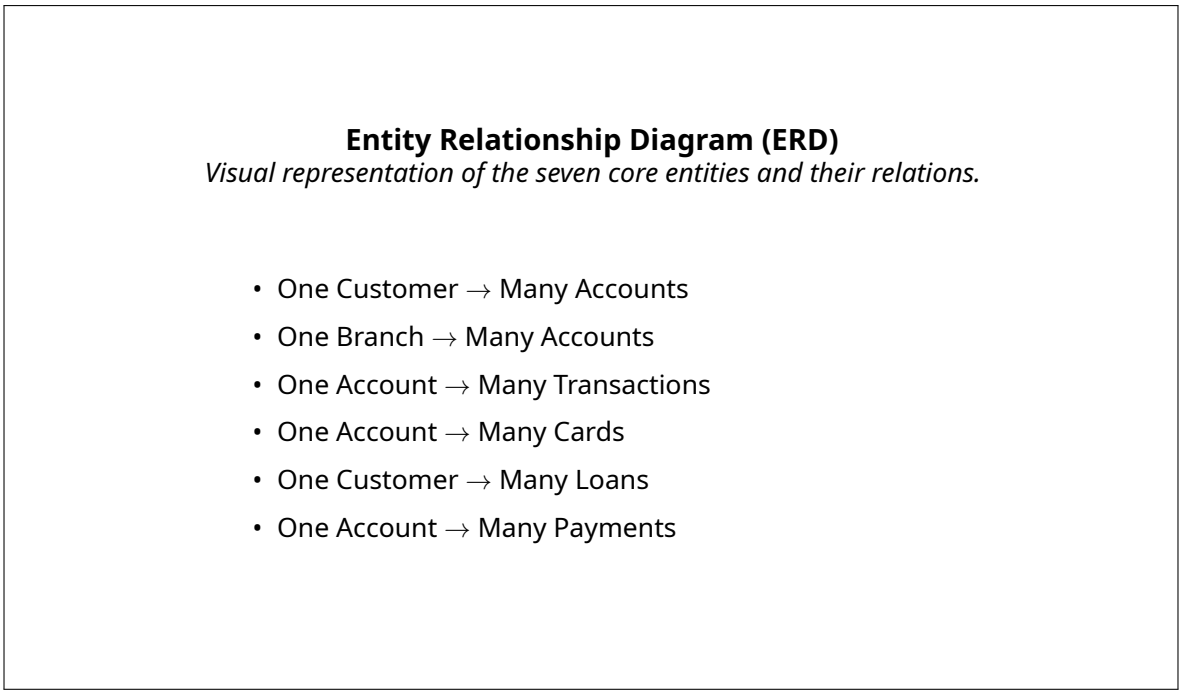

Figure 1: Conceptual Entity Relationship Model Overview

- **Transactions:** Captures the immediate pulse of money movement (deposits, withdrawals, and transfers). It utilizes a self-referencing mechanism (`related_account_id`) to cleanly track counterparty account details during fund transfers.
- **Cards:** Manages debit and credit credentials tied to accounts. *Note: CVV numbers are intentionally excluded from the schema to align with modern data privacy standards and security-by-design principles.*
- **Loans:** Maintains long-term credit products issued to customers, tracking principal capital, interest rates, and maturity dates.
- **Payments:** Dedicated to structured, higher-level outbound cash flows such as settling utility bills, credit card statements, or loan installments.

4 4. Engineering & Design Decisions

4.1 4.1 Fixed-Point Currency Representation (`INTEGER` Cents)

Floating-point arithmetic (`FLOAT`, `REAL`) introduces rounding errors that are unacceptable in financial applications. To guarantee absolute mathematical precision, all monetary metrics—including account balances, transaction volumes, and loan principles—are explicitly stored as `INTEGER` types representing cents (or minor currency units). For instance, a financial value of 150.75 TRY is processed as 15075. This design pattern completely avoids floating-point precision issues during complex aggregate calculations.

4.2 4.2 Data Integrity & Domain Constraints

Data quality is strictly enforced at the database level using a comprehensive array of schema constraints:

- **Domain Validations (`CHECK`):** Restricts business domains to valid states. For example, `account_type` is strictly bound to `checking` or `savings`. Similarly, `transaction_type` can only accept predefined entries like `deposit`, `withdrawal`, `transfer_in`, `transfer_out`, or `payment`.
- **Structural Safety:** Implements `NOT NULL`, `UNIQUE`, and proper `DEFAULT` timestamps to prevent orphan records or incomplete operational histories.

4.3 4.3 Decoupling Transactions from Payments

A major architectural choice was the explicit separation of the `transactions` and `payments` tables. `Transactions` represent raw, primitive cash ledger movements (e.g., pulling cash from an ATM or moving funds between accounts). `Payments` reflect business-level actions, such as paying down a loan balance or paying a recurring bill. This isolation prevents analytical confusion, simplifies the audit trail, and provides cleaner reporting vectors.

5 5. Synthetic Dataset Overview

To rigorously evaluate the schema and its constraints, a comprehensive, relational-compliant synthetic dataset was generated. The current data footprint distributed via `insert.sql` consists of:

Table 1: Synthetic Dataset Distribution

Target Table	Record Volume	Analytical Role
`branches`	8	Simulates regional corporate footprint.
`customers`	30	Provides a diverse customer demographic base.
`accounts`	45	Simulates multi-account customer behaviors.
`transactions`	160	Generates a high-density, multi-channel ledger history.
`cards`	20	Maps active plastic and digital payment methods.
`loans`	15	Simulates active risk and credit liabilities.
`payments`	30	Populates higher-level business payment actions.

6 6. SQL Implementation & Advanced Features

The system is divided into modular SQL scripts to ensure clean maintainability:

1. `schema.sql`: Establishes the physical tables, column structures, and constraints.
2. `insert.sql`: Hydrates the database with the synthetic operational dataset.
3. `views.sql`: Implements reusable abstractions for reporting.
4. `indexes.sql`: Optimizes performance for high-traffic query patterns.
5. `queries.sql`: Contains the business intelligence and analytical queries.

The implementation utilizes an array of powerful SQL features covered throughout the course, such as multi-stage `LEFT JOIN` operations, nested subqueries, dynamic data pivoting via conditional `CASE` expressions, and sophisticated date-time parsing.

7 7. Business Intelligence & Analytical Queries

The core value of the schema is demonstrated through eight distinct analytical queries located in `queries.sql`:

- **7.1 High-Value Customer Identification:** Joins `customers`, `accounts`, and `transactions` to rank users by total transaction volume and frequency, isolating the bank's highest-value clients.
- **7.2 Performance Analysis by Branch:** Compares performance across branches by calculating active accounts, transaction volumes, and financial flows. By applying a `LEFT JOIN`, empty or newly opened branches are preserved in reports instead of being filtered out.
- **7.3 Anomalous Activity & Outlier Detection:** Employs a `HAVING` clause backed by an aggregate subquery to flag accounts whose transaction volumes significantly exceed the global institutional average.
- **7.4 Channel Distribution Insight:** Evaluates consumer behavioral shifts across various access channels (ATM, mobile app, web portal, or physical branch counter).
- **7.5 Operational Risk Mitigation:** Extracts pending or failed transactions to provide system administrators with a real-time operational dashboard for technical issues or fraud mitigation.
- **7.6 Temporal Trend Analysis:** Uses SQLite date manipulation functions to group transaction frequencies into monthly buckets, exposing macro-level economic trends and seasonal behaviors.
- **7.7 Credit and Loan Lifecycle Monitoring:** Correlates active loans against payment behaviors to track collection rates and client repayment performance.
- **7.8 Advanced Customer Segmentation (CTE):** Utilizes a Common Table Expression to analyze individual user behavior, categorizing accounts into distinct cohorts: `high_activity`, `medium_activity`, and `normal_activity`.

8 8. Database Performance Optimization & Views

8.1 8.1 Reusable Reporting Views

To abstract query complexity from application layers and business analysts, three reporting views are built directly into `views.sql`:

- `v_active_accounts`: Provides an active account ledger enriched with basic customer profiles and overseeing branches.
- `v_customer_transaction_summary`: Aggregates lifelong transaction volumes per customer for CRM use cases.
- `v_branch_performance`: Offers corporate executives an overview of branch performance.

8.2 8.2 Indexing Strategy

To prevent full-table scans during heavy analytical joining operations, target indexes were placed on frequently filtered foreign keys via `indexes.sql`:

- `accounts(customer_id, branch_id)` for faster client-to-branch resolutions.
- `transactions(account_id, branch_id, transaction_date)` to boost transactional audit speeds.
- `payments(account_id)` and `loans(customer_id)` to speed up risk and credit tracking queries.

9 9. Project Demonstration Guide

The entire project workflow can be executed sequentially in any standard SQLite CLI interface or via DB Browser for SQLite. To set up the environment, run the scripts in the following exact order:

```
.read sql/schema.sql   -- 1. Initialize DB structures and constraints
.read sql/insert.sql   -- 2. Populate tables with synthetic records
.read sql/views.sql    -- 3. Abstraction views generation
.read sql/indexes.sql  -- 4. Apply indexing optimization strategies
.read sql/queries.sql  -- 5. Execute core business intelligence reports
```

This compilation process sets up the system, verifies data integrity rules, establishes optimization frameworks, and outputs all analytical query results onto the console window.

10 10. Repository Blueprint

The final repository layout is structured as follows:

```
banking-database-project/

   README.md               # Quick-start documentation & overview
   sql/                    # Modular database development files
       schema.sql          # Table DDL definitions and constraints
       insert.sql          # Sample data seed script
       queries.sql         # Analytical and reporting workloads
       views.sql           # Abstracted query layers
       indexes.sql         # Index structure definitions
   docs/                   # Academic documentation and reports
       final_report.pdf    # Compiled PDF version of this report
   images/                 # Visual asset directories
        erd.png            # Full Entity Relationship Diagram export
```

11 11. Conclusion

The **Banking Transaction & Account Management System** demonstrates the practical application of relational database principles in a high-concurrency industry. By converting unstructured operational activities into a deeply validated, normalized relational schema, the design guarantees data consistency across all customer interactions.

Furthermore, the implementation proves that an optimized database layer can handle complex analytical workloads—such as customer segmentation and risk modeling—efficiently. By leveraging advanced SQL features, the system provides a scalable foundation for modern, data-driven banking analytics.